

PVT 掌锐电子



一，音视频播放方式

关于视频或音频播放，一般有两种方式

1、使用MediaPlayer （比如本地音频视频播放器）

当使用Android原生的MediaPlayer，会使用NuPlayer 去进行文件的提取，音频和视频的时间对齐，音频和视频解码后输入的向下传递等操作

2、直接调用mediacodec处理音频和视频 （比如爱奇艺，carplay等）

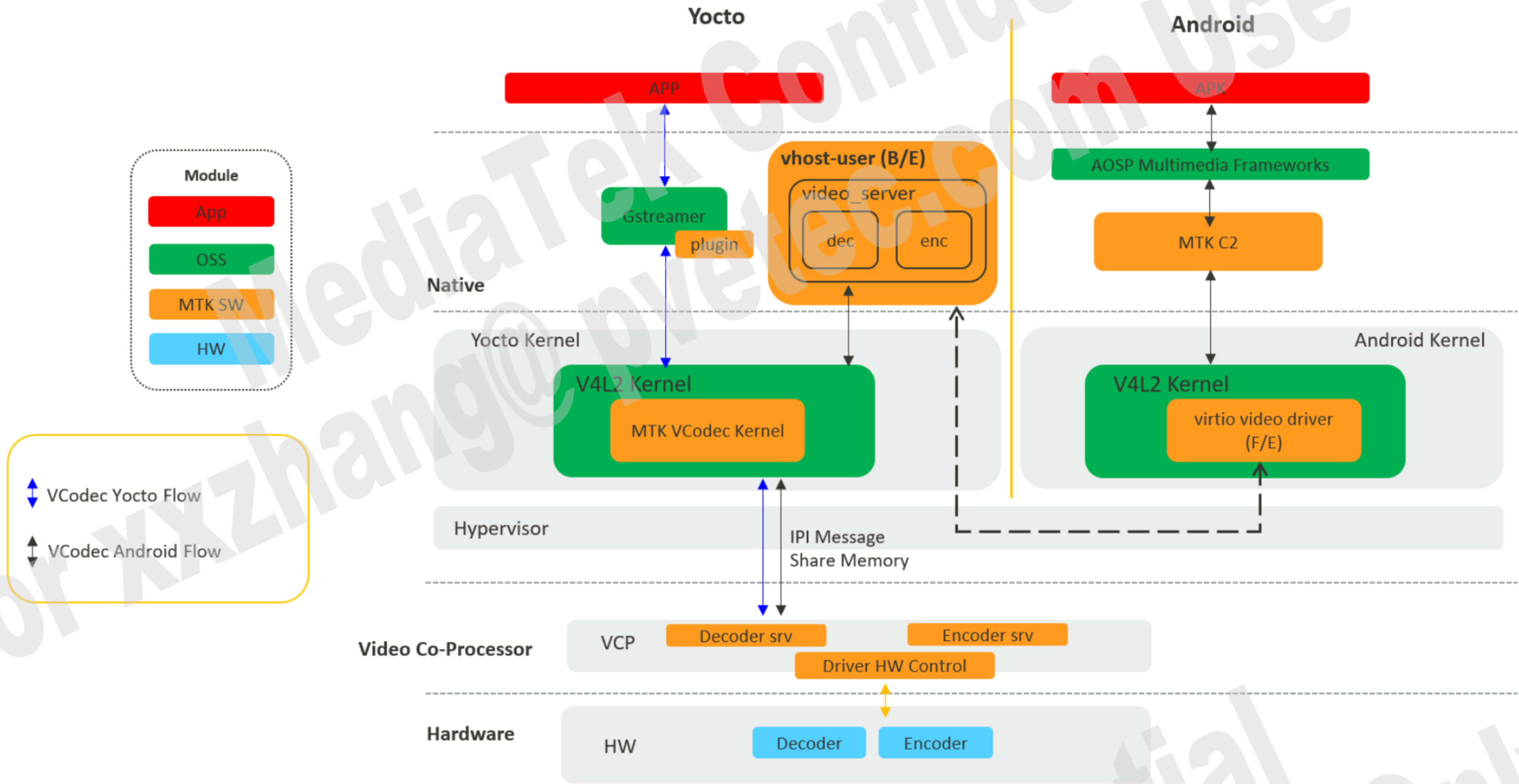
当使用mediacodec时，音频和视频都有app去处理。mediacodec只负责传入接收音视频原始数据，获取解码后的数据。其他操作由app进行。

二、8676 硬编解码性能

类别	编解码器	规格				
		分辨率	帧率	位速率	位深	属性/级别
解码器	H.264	4096x2176	60	160 Mbps	8/10	CBP,MP,HP, High 10 / 5.2
	H.265	4096x2176	60	160 Mbps	8/10	Main / 5.1 Main 10 / 5.1
	VP9	4096x2176	60	120 Mbps	8/10	Profile 0/2
	AV1	4096x2176	60	120 Mbps	8/10	Main/level5.1
编码器	H.264	3840x2160	60	160 Mbps	8/10	BP,MP,HP, High 10 / 5.2
	H.265	3840x2160	60	160 Mbps	8/10	Main, Main10 / 5.1
并发 (硬件性能)	-	4K60 VDEC + 4K60 VENC 或等效吞吐量				

表格标注最大支持能力，多路情况
通过表格大体计算获得，计算时，若3个
OS都在工作，场景都要考虑到。合并计算
超过性能最大值，可能播放失败或卡顿等

三、3OS软件架构



四、常见解码情况分析

使用MediaPlayer或Mediacodec判断开始和结束时间点

在遇到问题时，需要对Android的mainlog进行分析

若使用MediaPlayer时 会有NuPlayer和GenericSource的log信息

开始播放会看到这些信息，用于确定视频或音频开始

```
D NuPlayerDriver: NuPlayerDriver(0xb400007247873780) created, clientPid(7167)
```

```
D NuPlayer: onSetVideoSurface(0xb40000724919f800, no video decoder)
```

```
D GenericSource: FileSource remote
```

```
I GenericSource: start
```

NuPlayer结束播放时

```
1266 15877 D NuPlayerDriver: notifySeekComplete(0xb400007247873780)
```

```
1266 1911 D NuPlayerDriver: reset(0xb400007247873780) at state 5
```

```
1266 15877 D NuPlayerDriver: notifyResetComplete(0xb400007247873780)
```

```
1266 1911 D NuPlayerDriver: reset(0xb400007247873780) at state 0
```

对于mediacodec log

开始播放会有下面信息，用于确认开始播放时间

```
D MediaCodec: CreateByType: mime video/avc, encoder 0 pid 7167
```

```
D MediaCodec: [0xb40000723ba74000] init: CCodec 0xb4000072490250c0, CCodecBufferChannel 0xb40000723b9b6800
```

```
D MediaCodec: [0xb40000723ba74000] setState: 1
```

```
D MediaCodec: CreateByType: mime audio/mp4a-latm, encoder 0 pid 7167
```

```
D MediaCodec: [0xb400007247910000] init: CCodec 0xb40000724780d000, CCodecBufferChannel 0xb4000072478eac00
```

可以看到mediacodec音频一个线程，视频一个线程

通过查看MediaCodec 的setState状态来判断进度，状态6是正常播放中

五、使用硬编解码分析

对于8676或78，若是使用的视频硬解码，可能使用硬编解码场景和出现异常时更多。

硬编码用于录制视频文件，录制为H264或H265

硬解码用于使用AVC/HEVC等常见视频流的场景，比如本地播放时大多mp4文件，carplay等投屏工具，爱奇艺等播放软件

比如可以用这个确定播放和停止，也可以看到解码编码当时的数量

D C2MtkComponentStore: onNumberOfVideoInstancesChanged: Decoder 1, Encoder 1

D C2MtkComponentStore: onNumberOfVideoInstancesChanged: Decoder 0, Encoder 1

可以搜索 onNumberOfVideoInstancesChanged

若是视频解码看到 c2.mtk.XXX表示走的mtk的硬解码，若是音频或c2.android则是走的软解码

遇到问题，首先看mediaserver的进程，搜mediacodec，或nuplayer等

看到是硬解的话，搜索硬解码进程，可以搜c2mtk，进程号是硬解码的进程，全搜，对应开始或结束或问题时间点去查看异常定位问题。

D PipelineWatcher: [0xb4000072478eb0a8] pipelineFull: too many frames in pipeline (6)

这个一般不是异常，正常播放视频几乎都有这些信息，这个只能说明mediacodec处理

dequeueOutputbuffer

在等待处理数据

六、硬编解码分析中打开log和dump操作

当硬解播放视频或怀疑硬解码导致时，可以打开硬解码更多log

```
adb root
```

```
adb shell "setprop vendor.mtk.c2.enable.comp.log 1"
```

```
adb shell "setprop vendor.mtk.c2.enable.vdec.log 3"
```

```
adb shell "setprop vendor.mtk.c2.enable.vcodec.log 4"
```

adb后不能重启，重启上述命令会失效。adb 输入后，不断电继续抓取测试即可

```
adb shell "setprop vendor.mtk.c2.vdec.dump.input 1"
```

上面这条设1是c2mtk会dump硬解码的数据流，avc或hevc，保存到/data/vendor/vcodec下 input命名的bs文件中

可以pull出来，使用较新版的potplayer播放查看，设0关闭dump或重启机器不再dump

还可以dump 硬解码后的数据

```
adb shell "setprop vendor.mtk.c2.vdec.dump.output 1"，
```

保存到/data/vendor/vcodec下 output命名的文件中，此文件非mtk无权打开访问，需要给mtk查看，mtk说是mtk私有的加密数据，设0关闭解码后dump

硬编码命令

```
encoder
```

```
adb root
```

```
adb shell setprop vendor.mtk.c2.enable.venc.log 3
```

```
adb shell setprop vendor.mtk.c2.enable.comp.log 3
```

开启后，可以看周期input和output，搜索这个

NotifyOutputBufferReady //视频帧解码后output解码完成时间戳信息

process: input // 视频帧每一帧input时间戳信息

七、常见log及分析

遇到问题，首先看mediaserver的进程，搜mediacodec，或nuplayer等

看到是硬解的话，搜索硬解码进程，可以搜c2mtk，进程号是硬解码的进程，全搜，对应开始或结束或问题时间点去查看异常定位问题。

D PipelineWatcher: [0xb4000072478eb0a8] pipelineFull: too many frames in pipeline (6)

这个一般不是异常，正常播放视频几乎都有这些信息，这个只能说明mediacodec处理 dequeueOutputbuffer在等待处理数据

视频出现卡顿，一般有两种可能，一种是分辨率过高，解码吃力。在8676和8678上比较少。

硬解码的能力能到4K 60fps，软解可能只有1080P 30或60fps

还有更常见的可能，是解码后数据不能及时送出去，音频和视频下行通道堵塞，都会引起卡顿。

常见的log能看到wait fence

“ wait fence ” 是vdec hal在等待 framebuffer上的 fence信息（vdec和 display之间的sync机制），所以这种time out通常是 display或者 PQ 处理不及时，导致framebuffer 循环卡主的 播放卡顿，音频下行链路卡住可能也会打印。Mtk说少量打印一般问题不大，频繁打印就需要考虑显示有卡顿

若怀疑送显有卡顿，尝试输入下面的cmd再验证下是否能复制到问题：

```
adb shell service call SurfaceFlinger 1008 i32 1 //关闭PQ
```

复现问题时的 log 和时间点

记录正常界面与异常界面的时间点。

导出MTKLOG，与前面dump出的数据一并提供用于分析。

an_error ged handle 这个不是异常，几乎每次播放都有

如果使用了硬解码，可能mainlog里看不到异常。可以简单的确定kernel log里有没有异常，搜索 vdec|utc，在sos中搜kernel和vcp log

utc时间可能没有和mainlog一致，那就kernel utc+8 = mainlog时间

看看vdec相关有没有在对应时间点有没有fail或error提示，来确定是底层故障。

还有一个vcplog也可以搜索，也是搜vdec，vcplog里没有utc打印，时间需要跟kernel去对齐

八、常见log及分析

遇到常见问题log:

```
NuPlayerRenderer: onNewAudioMediaTime updateAnchor: mediaTimeUs 30304000 delta[262212]us,  
pendingDuration 1280000 us
```

这段日志信息反映了NuPlayerRenderer在音频同步过程中更新锚点时间(updateAnchor)的关键参数，具体分析如下：
参数含义解析：

mediaTimeUs 30304000：表示当前音频媒体时间戳（单位微秒），即音频流播放到的位置。

delta[262212]us：指当前媒体时间与前一锚点时间的差值，用于计算时间偏差。

pendingDuration 1280000us：待处理的音频数据缓存时长（1.28秒），反映缓冲区数据积压情况。

我理解是，当解码后音频向下通路出现堵塞，会出现pendingDuration 积累增加的情况

当视频解码出来不及时时，也会出现这个，相当于音频在等待视频做时间同步

遇到常见问题log:

```
NuPlayerRenderer: we're late dropping one timeUs 31583 ms after 13 frames
```

该日志表明NuPlayerRenderer在视频渲染过程中出现严重延迟，导致连续丢弃13帧后仍存在31.583秒的同步滞后，属于严重性能异常。以下是关键分析与解决方案：

同步机制崩溃

视频帧时间戳（timeUs 31583ms）与系统时钟偏差超过阈值，触发强制丢帧策略1。

连续13帧未能及时渲染，说明同步补偿机制已失效，可能因音频主时钟异常或解码阻塞引发2。

硬件/驱动瓶颈

高延迟常见于GPU渲染管线阻塞（如SurfaceFlinger的dequeueBuffer超时100ms以上）

九、常见log及分析

76/78

```
CCodecBuffers: [c2.mtk.avc.encoder#507:Output[N]] pushToStash: pushToStash -- pending size = 83
BufferPoolAccessor2.0: bufferpool2 0xb40000743840d828 : 170(84999660 size) total buffers -
170(84999660 size) used buffers - 26257/26427 (recycle/alloc) - 383/26413 (fetch/transfer)
```

若遇到c2hal media 内容占用高的情况，比如上面信息log

重点看 total buffers和used buffers是不是比较大，或一直在增长

多个客户都遇到过，最后查到原因是PC投屏软件导致，投屏会使用soc的编码功能，投屏有时候不能正常退出，退出时没有关闭编码，导致编码一直在生成，而没有消费者取走导致。

投屏不是正常使用场景，投屏软件都是第三方的，只能规避，要注意这一点

在使用mediaplayer进行播放时，客户经常会遇到在app里收到 -19 -38等 mediaplayerNative的消息
很多时候在初始化配置surface或后台恢复时，surface配置或一些时序逻辑不对导致。
最好的办法是，看看mtk原生的gallery的实现逻辑

十、卡顿或黑屏等显示问题dump

遇到播放黑屏或卡顿显示问题的话，很多时候跟surface或hwc或PQ相关
需要看下面的信息定位，很多时候显示问题log很难看到异常，需要保存现场dump实验测试验证。

下次再复现需要保持现场输入下面命令测试或dump数据

1, PQ bypass看看问题能不能复现。 指令如下：

```
adb root
```

```
adb shell dumpsys vendor.mediatek.hardware.pq_aidl.IPictureQuality_AIDL/default --set_pq_persist_property "persist.vendor.sys.pq.disp.color.bypass" 1
adb shell dumpsys vendor.mediatek.hardware.pq_aidl.IPictureQuality_AIDL/default --set_pq_persist_property "persist.vendor.sys.pq.disp.ccorr.bypass" 1
adb shell dumpsys vendor.mediatek.hardware.pq_aidl.IPictureQuality_AIDL/default --set_pq_persist_property "persist.vendor.sys.pq.disp.gamma.bypass" 1
adb shell dumpsys vendor.mediatek.hardware.pq_aidl.IPictureQuality_AIDL/default --set_pq_persist_property "persist.vendor.sys.pq.disp.dither.bypass" 1
adb shell dumpsys vendor.mediatek.hardware.pq_aidl.IPictureQuality_AIDL/default --set_pq_persist_property "persist.vendor.sys.pq.disp.aal.bypass" 1
adb shell dumpsys vendor.mediatek.hardware.pq_aidl.IPictureQuality_AIDL/default --set_pq_persist_property "persist.vendor.sys.pq.disp.c3d.bypass" 1
adb shell dumpsys vendor.mediatek.hardware.pq_aidl.IPictureQuality_AIDL/default --set_pq_persist_property "persist.vendor.sys.pq.disp.tdshp.bypass" 1
```

如果问题依就存在的话， 帮着使用附件中的脚本抓取些数据， 操作如下：

a, 执行下面的脚本。

```
adb root
```

```
adb shell "echo mobile:on > /proc/mtkfb"
```

```
adb shell "echo detail:on > /proc/mtkfb"
```

```
adb shell setenforce 0 &&adb shell setprop persist.vendor.debug.hwc.log V&& adb shell setprop vendor.debug.hwc.skip_log 0
```

b, 分别在复现后问题的界面中在PC机中执行脚本。 [hwc_dump.zip\(\)](#)

暂时无法在飞书文档外展示此内容

先运行： 0_setup.bat 再运行： 1_dump.bat 然后， 再将DUMP出来的数据目录SF_dump打包给我们用于分析。 同时在对对应界面中dumpsys SurfaceFlinger用于分析。 操作如下： adb shell dumpsys SurfaceFlinger > sf_ok.txt或sf_ng.txt

十一、gst-launch-1.0播放测试常见使用

yocto gst 播放视频 gst-play.c

(1) adb push xxx /data/ push一个video文件到平台，比如xxx.mp4 push到data 目录

(2) source /data/wayland_env_file

(3) gst-launch-1.0 playbin flags=0x42 video-sink="v4l2convert output-io-mode=dmabuf-import capture-io-mode=dmabuf disable-passthrough=true propose-allocation=3 name=mtkmdp ! video/x-raw,format=BGRA,width=640,height=360,pixel-aspect-ratio=1/1 ! waylandsink" audio-sink="audioconvert ! audioresample ! audio/x-raw,rate=48000,format=S16LE ! alsasink device=hw:0,6" uri=file:///data/xxx

amixer cset name='ADDA_DL_CH1 DL6_CH1' 1

amixer cset name='ADDA_DL_CH2 DL6_CH2' 1

amixer cset name='ADDA_DL_CH3 DL6_CH1' 1

amixer cset name='ADDA_DL_CH4 DL6_CH2' 1

amixer cset name='HPL Mux' 'Audio Playback'

amixer cset name='HPR Mux' 'Audio Playback'

amixer cset name='Ext_Speaker_Amp Switch' 1

/data/gst-play-1.0 --flags=0x42 --videosink="v4l2convert output-io-mode=dmabuf-import capture-io-mode=dmabuf disable-passthrough=true propose-allocation=3 name=mtkmdp ! video/x-raw,format=BGRA,width=640,height=360,pixel-aspect-ratio=1/1 ! waylandsink" --audiosink="audioconvert ! audioresample ! audio/x-raw,rate=48000,format=S16LE ! alsasink device=hw:0,6" /data/xxx.mp4 试下

还是不行的话，拿掉 --audiosink="audioconvert ! audioresample ! audio/x-raw,rate=48000,format=S16LE ! alsasink device=hw:0,6"看下

source /data/wayland_env_file

layer-add-surfaces -l 1 -s 1000 -d DP-2 &

export GST_DEBUG=6 GST_DEBUG_COLOR_MODE=off GST_DEBUG_FILE=/tmp/gst.log

gst-launch-1.0 playbin flags=0x42 video-sink="v4l2convert output-io-mode=dmabuf-import capture-io-mode=dmabuf disable-passthrough=true propose-allocation=3 name=mtkmdp ! video/x-raw,format=BGRA,width=1920,height=720,pixel-aspect-ratio=1/1 ! waylandsink" uri=file:///data/input.mp4

PVT

Thank You

www.pvetec.com